

Algoritmer & data strukturer

Uge 5-6: Recursion og array-sortering

Alexander Bak Thorsen, Viktor Kirkeby Førreisdahl
Bertram Agerskov, Jonas Harwig Mejlholm

5. oktober 2026

I vores analyser i opgaverne følger vi disse steps:

The Recursion Recipe

Everything we have done so far, in four steps.

1. State the problem in terms of its size or complexity.
2. Find the **base case**, and show it can be solved without recursion.
3. Find the **recursive case**: assume the smaller problem is solved, and use it.
4. Check that the recursive case makes **progress** and reaches the base case.

Analyse, design, *then* code. You will be asked to write these four down.

Figur 1: Recursion steps

Øvelse 1

(A) Søg rekursivt efter en værdi i et array

Step 1

Størrelsen på vores problem er størrelsen på vores array, da hele vores array muligvis skal gennemgås for at finde det ønskede element, medmindre det findes før arrayet er gennemgået fuldstændig. Ergo gætter vi på, at vores kompleksitet bliver $O(N)$.

Step 2

En base case er vores måde at returnere fra en rekursiv funktion, så vi ikke ender i et uendeligt loop. Når basecasen udføres, vil vores rekursive funktion derved ikke blive kaldt igen, og derved slutter hele funktionen.

I vores tilfælde skal vi have vores base cases enten når vi har gennemløbet hele vores array uden at finde det ønskede element:

```
1  if (size == 0)
2  {
3      return false;
4  }
```

Code Block 1: Base case 2 ved fuld gennemløbning af array

Eller når det givne element bliver fundet:

```
1  if (arr[0] == target)
2  {
3      return true;
4  }
```

Code Block 2: Base case 1 ved fund af element

Derved har vi fundet de to base cases, som skal bruges i vores rekursive funktion.

Step 3

Den rekursive funktion løser vores problem med et funktionskald af gangen og ved at antage, at det forrige problem er løst. Altså skal funktionen igennem vores array lige så stille og kalde den rekursive funktion, indtil man rammer en af vores base cases. På den måde sikrer man, at funktionen laver fremskridt og får et mindre input hver gang den bliver kaldt, og til sidst rammes vores base case.

Ud fra denne tilgang kan vi danne følgende rekursive funktion for at kunne finde element x i array A ved først at tjekke, om nogle af vores base cases er ramt. Ellers kalder vi funktionen igen med en mindre size og inkrementerer pladsen i vores array med en og sender target videre igen som parametre:

```
1  bool search(int *arr, int size, int target)
2  {
3      if (size == 0)
4      {
5          return false;
6      }
7
8      if (arr[0] == target)
9      {
10         return true;
11     }
12
13     // Til step 4 test:
14     // std::cout << "arr[0] = " << arr[0] << " size = " << size << std::endl;
15 }
```

```

16     return search(arr + 1, size - 1, target);
17 }

```

Code Block 3: Rekursiv funktion til at finde element x

Step 4

For at tjekke, om vores rekursive funktion laver fremskridt, udprintes vores arr samt size hver gang, vores rekursive funktion bliver kaldt. Vi har følgende array, hvor vores funktion fra "Rekursiv funktion til at finde element x" bruges til at tjekke om det givne element er der:

```

1     #define TAL_VI_LEDER_EFTER 7
2
3     int array[10] = {-89, 1, 2, 3, 4, 200, 200000, 213123, 132323, 10000000};
4
5     std::cout << "findes tallet " << TAL_VI_LEDER_EFTER << " i vores array? " << std::endl << std::boolalpha
        << search(array, std::size(array), TAL_VI_LEDER_EFTER);

```

Code Block 4: Array, som sendes til search funktion

Hvilket producere følgende output:

```

findes tallet 7 i vores array?
arr[0] = -89 size = 10
arr[0] = 1 size = 9
arr[0] = 2 size = 8
arr[0] = 3 size = 7
arr[0] = 4 size = 6
arr[0] = 200 size = 5
arr[0] = 200000 size = 4
arr[0] = 213123 size = 3
arr[0] = 132323 size = 2
arr[0] = 10000000 size = 1
false

```

Figur 2: OUTPUT fra search funktion

Derved kan vi konstatere, at vores rekursive funktion laver fremskridt og til sidst stopper ved den base case, hvor $size == 0$, da vores target element ikke blev fundet. Det kan også ses til sidst på outputtet, at vi udskriver *false*, da 7 ikke findes i vores array.

Tidskompleksitet i worst case

Ved worst case, som kan ses på OUTPUT fra search funktion, kalder vi vores funktion uden at finde det ønskede element, hvilket returnerer "false" efter at alle elementer er blevet tjekket igennem. Derved kan vi konkludere, at tidskompleksiteten bliver til $O(N)$, da den afhænger af mængden af elementer, som vores array indeholder.

(B) Find største og mindste element rekursivt

Vi følger samme trin, som i opgave A og derfor uddybes der ikke så meget i teksten, som i opgave A

Step 1

Størrelsen på vores problem er størrelsen på vores array, da hele vores array, skal gennemgås for at tjekke for det mindste og det største element. Ergo kan vi sige vi allerede nu konstaterer, at vores kompleksitet bliver $O(N)$.

Step 2

Da hele vores array skal gennemgås, skal alle elementer i `arr[0]` sammenlignes med det største og mindste element i resten af arrayet. Derved ender vi i begge funktioner kun med at have en base case, som begynder at returnere og tjekke, når vores size bliver 1.

```
1     if (size == 1)
2     {
3         return arr[0];
4     }
```

Code Block 5: Base case 1 ved både max og min funktioner

Step 3

Vi danner en ny variabel inden i vores funktion kaldet "maxOfRest", som har til opgave at finde max af resten af vores array. Denne bruger vores allerede definerede maxfunktion og derved dannes de rekursive funktionskald. Disse kaldes hele vejen indtil vi rammer base casen, hvori vi begynder at returnere lige så stille. Herved tjekker vi, om det niveau på `arr[0]`, vi står på, er større end "maxOfRest". Er dette tilfældet, sendes `arr[0]` videre op ad trinnet og sammenlignes igen på næste niveau, indtil vi til sidst har gennemgået alle elementer. For at sikre, at vores max-funktion ikke kaldes på et tomt array, bruger vi først "assert", som tillader koden at fortsætte, hvis udtrykket er true. Der sendes en fejlbesked ud, hvis size nogensinde rammer 0 eller under. Vi tager to parametre: en pointer til vores array og størrelsen af vores array. Vores funktion kommer dermed til at se ud som følger:

```
1 int max(int *arr, int size)
2 {
3     assert(size > 0); // Stopper funktionen, hvis foelgende er false og returnerer en fejlbesked. Der kan
4                       // nemlig ikke findes en max af et tomt array
5
6     if (size == 1)
7     {
8         return arr[0];
9     }
10
11     int maxOfRest = max(arr + 1, size - 1);
12
13     if (arr[0] > maxOfRest)
14     {
15         return arr[0];
16     }
17     return maxOfRest;
18 }
```

Code Block 6: Rekursiv max funktion

Derved kaldes vores maxfunktion på følgende måde:

```
1 std::cout << "Max i vores array er: " << max(array, std::size(array));
```

Code Block 7: Kald af rekursiv max funktion

Step 4

For at tjekke, om vores rekursive funktion laver fremskridt, udprintes vores arr, samt size og "maxOfRest" hver gang, vores rekursive funktion er på vej tilbage. Vi tester på samme array som i opgave A, hvor vi kalder "Kald af rekursiv max funktion". Dermed bliver Max kaldt så mange gange, at vi når op til det sidste element i vores array, og vi udprinter derefter, når vi er på vej tilbage. Derved kan vi se, at vi starter med at have "maxOfRest" på 10000000 og derefter tjekkes det med forrige trins element osv:

```
Max i vores array er:
arr[0] = 132323 size = 2 maxOfRest: 10000000
arr[0] = 213123 size = 3 maxOfRest: 10000000
arr[0] = 200000 size = 4 maxOfRest: 10000000
arr[0] = 200 size = 5 maxOfRest: 10000000
arr[0] = 4 size = 6 maxOfRest: 10000000
arr[0] = 3 size = 7 maxOfRest: 10000000
arr[0] = 2 size = 8 maxOfRest: 10000000
arr[0] = 1 size = 9 maxOfRest: 10000000
arr[0] = -89 size = 10 maxOfRest: 10000000
10000000
```

Figur 3: OUTPUT max funktion

Det er nu tydeligt, at vores funktion gennemgår alle elementerne i vores array og undervejs sender det største element videre til næste trin. Samme logik er brugt for *min*-funktionen, som kan ses nedenunder:

```
1 int min(int *arr, int size)
2 {
3     assert(size > 0); // Stopper funktionen, hvis foelgende er false og returnerer en fejlbesked. Der kan
4     nemlig ikke findes en min af et tomt array
5
6     if (size == 1)
7     {
8         return arr[0];
9     }
10    int minOfRest = min(arr + 1, size - 1);
11
12    if (arr[0] < minOfRest)
13    {
14        return arr[0];
15    }
16    return minOfRest;
17 }
```

Code Block 8: Rekursiv min funktion

Test af min funktion på samme array som før:

```
Min i vores array er:  
arr[0] = 132323 size = 2 minOfRest: 10000000  
arr[0] = 213123 size = 3 minOfRest: 132323  
arr[0] = 200000 size = 4 minOfRest: 132323  
arr[0] = 200 size = 5 minOfRest: 132323  
arr[0] = 4 size = 6 minOfRest: 200  
arr[0] = 3 size = 7 minOfRest: 4  
arr[0] = 2 size = 8 minOfRest: 3  
arr[0] = 1 size = 9 minOfRest: 2  
arr[0] = -89 size = 10 minOfRest: 1  
-89
```

Figur 4: OUTPUT min funktion

Tidskompleksitet i worst case

Som vi kan se på OUTPUT max funktion og OUTPUT min funktion gennemgås alle elementer i vores array, og derved ender tidskompleksiteten på $\Theta(N)$ ved både worst og best case, som også var det, vi fandt frem til i step 1.

Øvelse 2

(A) Udskriv et trekantsmønster rekursivt

Funktionen modtager m og n og udskriver linjer med m til n stjerner og derefter tilbage til m . Linjen med n stjerner udskrives to gange.

Step 1

Størrelsen af vores problem kommer til at fylde $(n - m + 1) \cdot 2$ linjer, da vi skal udskrive den mængde af linjer med stjernetegne fra m til n og tilbage igen til m .

Derved bliver tidskompleksiteten $O(n - m)$, da man i BIG O notation ignorerer konstanter. Størrelsen derimod bliver antallet af linjer, som vores program kommer til at udskrive.

Step 2

I denne rekursive funktion skal vi ikke returnere noget, så denne laves til en *void*, og når vi har med *void*-funktioner at gøre, vil vores base-case være det sted, hvor vores rekursive funktion ikke kalder sig selv længere. Ergo går man ligesom den anden vej igen efter base casen er blevet ramt. Altså vil vi efter vores base case færdiggøre alle de andre rekursive funktionskald, som vi har kaldt imens $m < n$. Dermed ender vores basecase med at se sådan ud, da vi i vores funktion udprinter stjerner undervejs, men skal sørge for at printe to rækker med n stjerner:

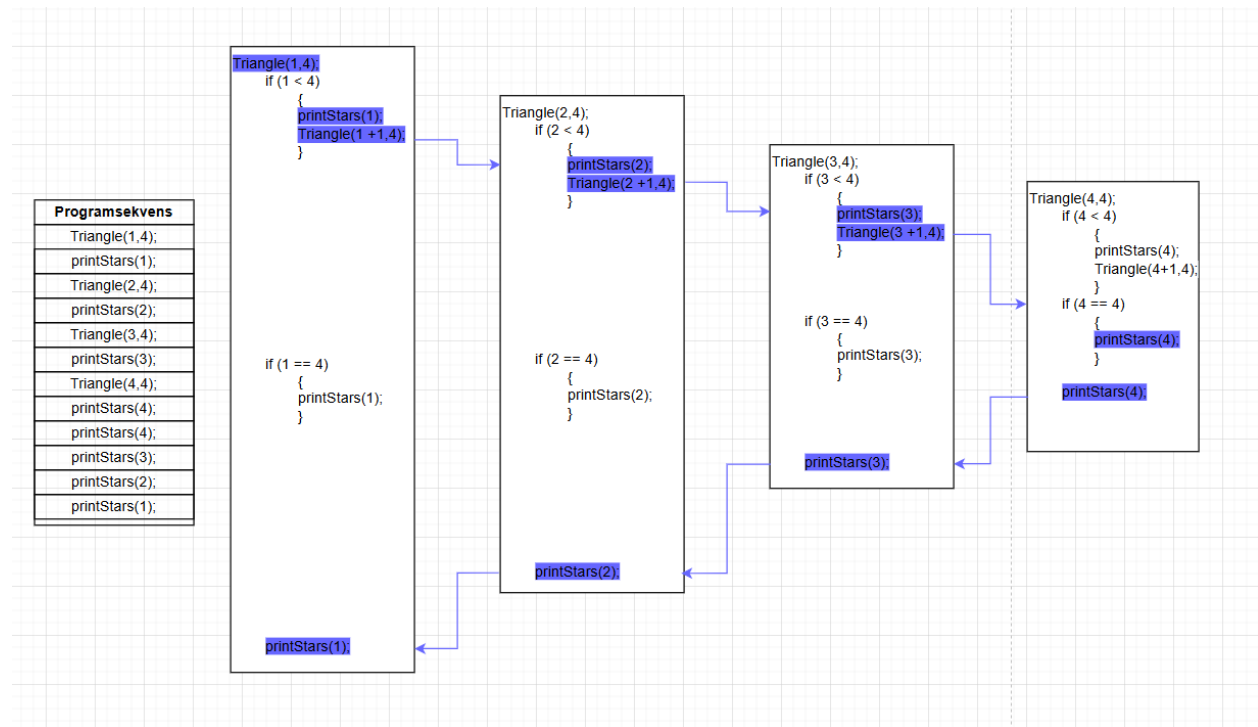
```
1  if (m == n)
2  {
3      printStars(m);
4  }
```

Code Block 9: Void base case

Vi har også en *printStars* i bunden af vores funktion, som udprinter samme antal stjerner som i vores base case. Derfor er der kun et funktionskald til *printStars* inde i base casen.

Step 3

Når vi har vores base case på plads, kan vi danne en rekursiv void-funktion, som pakker vores base case ind imellem to udprintninger af stjerner. Derved printer vi først m antal stjerner ud og derefter tjekker vi om $m < n$. Er dette tilfældet kaldes vores rekursive funktion igen, men med $m+1$ indtil vi rammer vores basecase. Vi danner også en hjælpefunktion *printStars(int s)*. Efter basecasen tegnes der igen stjerner, som passer med antallet m , da denne tegnes på sidste linje af vores funktion af alle de andre rekursive funktioner, som ligesom returnerer og udprinter. Vi har tegnet hele funktionskaldet i draw.io for at skabe et visuelt overblik, hvor vores program ligesom følger den blå farve. Her fremvises det, at vi har lavet 4 kald til triangle, hvoraf 3 er rekursive, og når de så returnerer, vil de tegne det ønskede antal stjerner.



Figur 5: Draw.io visualisering

Ud fra vores visualisering kan vi danne følgende rekursive funktion:

```

1 void triangle(int m, int n)
2 {
3     if (m < n)
4     {
5         printStars(m);
6         triangle(m + 1, n);
7     }
8
9     if (m == n)
10    {
11        printStars(m);
12    }
13
14    printStars(m);
15 }

```

Code Block 10: Rekursiv triangle funktion

Hvor *printStars(m)* ser sådan ud:

```

1 void printStars(int s)
2 {
3     for (auto i = 0; i < s; i++)
4     {
5         std::cout << "*";
6     }
7     std::cout << std::endl;
8 }

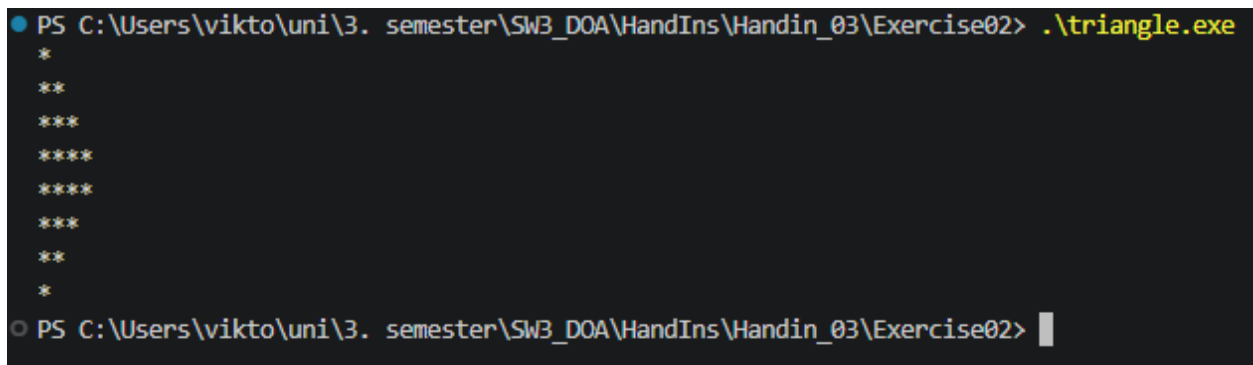
```

Code Block 11: Hjælpefunktion *printStars()*

Da vi i opgavebeskrivelsen angiver, at n skal være større end m , og vi derefter skal bruge $m + 1$, indtil vi når n , er der ikke lavet nogen logik, som håndterer det tilfælde, hvis $m > n$, når vi kalder vores funktion.

Step 4

Vi tjekker vores implementerede rekursive funktion med parametrene (1,4) og får følgende output:



```

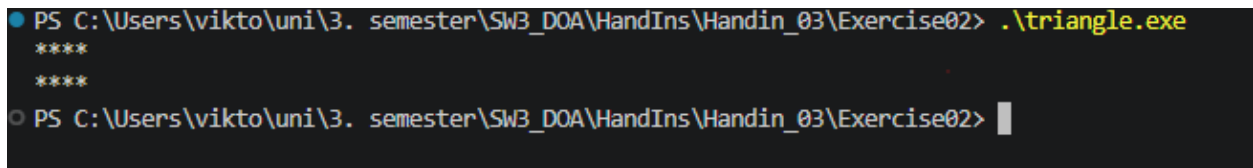
PS C:\Users\vikto\uni\3. semester\SW3_D0A\HandIns\Handin_03\Exercise02> .\triangle.exe
*
**
***
****
****
***
**
*
PS C:\Users\vikto\uni\3. semester\SW3_D0A\HandIns\Handin_03\Exercise02>

```

Figur 6: OUTPUT fra *triangle(1,4)*

Her kan vi se, at m inkrementeres med 1 for hvert rekursivt kald, indtil $m == n$ og der udprintes to linjer med samme antal stjerner, hvorefter funktionen går tilbage igen og færdiggør sine kald, som det kan ses i Draw.io visualisering.

Der laves endnu en test med $n == m$, som gerne skulle udprinte to linjer med samme antal stjerner:



```

PS C:\Users\vikto\uni\3. semester\SW3_D0A\HandIns\Handin_03\Exercise02> .\triangle.exe
****
****
PS C:\Users\vikto\uni\3. semester\SW3_D0A\HandIns\Handin_03\Exercise02>

```

Figur 7: OUTPUT fra *triangle(4,4)*

Tidskompleksitet i worst case

Koden udprinter $(n - m + 1) * 2$ linjer, da vi tæller op og ned fra m til n . Derved bliver tidskompleksiteten $O(n - m)$ målt i antal linjer og funktionskald.

Øvelse 3

(A) Udskriv sidernes fordeling ved booklet printing

Implementer `bookletPrint(startPage, endPage)` rekursivt, så hver arkudskrift angiver de fire sider på arket. Antallet af sider antages at være deleligt med 4. For siderne 1 til 8 skal første ark indeholde 1, 2, 7, 8 og andet ark 3, 4, 5, 6.

```
1  #include <iostream>
2
3  void bookletPrintHelp(int startPage, int endPage, int sheetnumber) // Ekstra funktion lavet, saa der kan
   taelles et sheetnummer (sheetnumber)
4  {
5      std::cout << "Sheet " << sheetnumber << " contains pages "
6          << startPage << ", " << startPage + 1 << ", " << endPage - 1 << ", " << endPage << "\n"; //
       Printer antallet af brugte sheets og siderne printet paa papiret
7
8      if (endPage - startPage < 4) // Svarer til, at basecasen er klaret og printet tidligere, og funktionen maa
       derfor stoppe
9      {
10         return;
11     }
12
13     bookletPrintHelp(startPage + 2, endPage - 2, sheetnumber + 1); // Kalder den recursive case, hvor den
       bruger de nye sidetal og et opdateret sheetnummer
14 }
15
16 void bookletPrint(int startPage, int endPage) // Den originale funktion, der skulle laves
17 {
18     bookletPrintHelp(startPage, endPage, 1); // Kalder den ekstra hjaelfunktion med sheetnumber
19 }
20
21 int main()
22 { // Tester forskellige vaerdier
23     bookletPrint(1, 8);
24     bookletPrint(1, 12);
25     bookletPrint(1, 4);
26 }
```

Code Block 12: Rekursiv booklet printing

Øvelse 4

(A) Find en vej gennem en maze rekursivt

Søg fra position (1,1) efter feltet E i et array med ROWS rækker og COLS kolonner. X markerer vægge, og blanke felter kan passeres. Returner true, hvis der findes en vej, ellers false.

Step 1

Størrelsen på problemet kan beskrives som antallet af felter i mazen som endnu ikke er besøgt. Mazon indeholder maksimalt $ROWS \cdot COLS$ felter.

Step 2

Funktionen har 3 base cases.

1. base case opstår hvis den aktuelle position ligger uden for mazens grænse og ikke kan føre til exit.

```
1 if (row < 0 || row >= ROWS ||  
2   col < 0 || col >= COLS)  
3 {  
4   return false;  
5 }
```

Code Block 13: 1. base case

2. base case opstår hvis positionen indeholder en væg X eller allerede er blevet tjekket V.

```
1 if (maze[row][col] == 'X' ||  
2   maze[row][col] == 'V')  
3 {  
4   return false;  
5 }
```

Code Block 14: 2. base case

Sidste base case opstår når exit er fundet og positionen indeholder E.

```
1 if (maze[row][col] == 'E')  
2 {  
3   return true;  
4 }
```

Code Block 15: 3. base case

Step 3

Når ingen af base cases rammes, ligger den aktuelle position på et frit felt i mazen.

Feltet markeres som tjekket:

```
1 maze[row][col] = 'V';
```

Herefter kaldes funktionen rekursivt for de fire mulige felter: op, ned, venstre og højre.

De rekursive funktionskald forbindes med en OR-operator || - så hvis bare en af vores funktionskald rammer E og returnerer true, vil hele funktionen returnere true.

```
1 bool findPath(char maze[ROWS][COLS], int row, int col)  
2 {  
3   if (row < 0 || row >= ROWS ||  
4       col < 0 || col >= COLS)
```

```

5     {
6         return false;
7     }
8
9     if (maze[row][col] == 'X' ||
10        maze[row][col] == 'V')
11     {
12         return false;
13     }
14
15     if (maze[row][col] == 'E')
16     {
17         return true;
18     }
19
20     maze[row][col] = 'V';
21
22     return findPath(maze, row - 1, col) ||
23            findPath(maze, row + 1, col) ||
24            findPath(maze, row, col - 1) ||
25            findPath(maze, row, col + 1);
26 }

```

Code Block 16: Rekursiv funktion

Step 4

For at sikre funktionen laver fremskridt markeres hvert tjekkede felt med V. Hvis et tjekket felt igen rammes af et rekursivt funktionskald vil base case 2 returnere **false**. Antallet af ikke-tjekkede felter bliver derved mindre, som algoritmen kører igennem mazen.

Eftersom mazen kun kan indeholde $ROWS \cdot COLS$ felter kan rekursionen ikke forsætte uendeligt. Funktionen vil enten ramme mazens grænser, en væg X, et allerede tjekket felt V og returnere **false**. Til sidst finder den enten E og returnere **true**, eller returnere **false**, hvis det ikke er muligt at finde E.

For at teste funktionen anvendes først mazen fra opgavebeskrivelsen:

```

char maze1[ROWS][COLS] = {
{'X','X','X','X','X'},
{'X',' ',' ',' ','X'},
{'X',' ','X',' ','X'},
{'X',' ','X',' ','X'},
{'X',' ','X',' ','X'},
{'X','E','X','X','X'}
};

```

Denne maze indeholder en valid vej fra (1,1) til E og bør returnere **true**.

Herefter ændres mazen, så vejen til E er blokeret:

```

char maze2[ROWS][COLS] = {
{'X','X','X','X','X'},
{'X',' ',' ',' ','X'},
{'X','X','X',' ','X'},
{'X',' ','X',' ','X'},
{'X','E','X','X','X'}
};

```

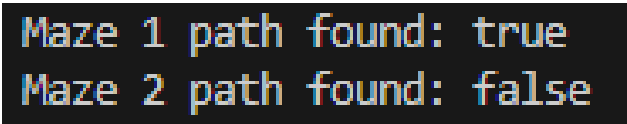
Denne maze indeholder ikke en valid vej til E og bør returnere **false**.

Vi tester begge i main():

```
1  std::cout << std::boolalpha;
2
3  std::cout << "Maze 1 path found: "
4              << findPath(maze1, 1, 1)
5              << std::endl;
6
7  std::cout << "Maze 2 path found: "
8              << findPath(maze2, 1, 1)
9              << std::endl;
```

Code Block 17: Test af rekursiv funktion

Testen giver følgende output:



```
Maze 1 path found: true
Maze 2 path found: false
```

Figur 8: Output rekursiv funktion

Vi kan derfor konstatere at funktionen både kan finde vej til E, og korrekt returnere `false` hvis ingen vej findes.

```
1  #include <iostream>
2  constexpr int ROWS = 5;
3  constexpr int COLS = 5;
4
5  bool findPath(char maze[ROWS][COLS], int row, int col)
6  {
7      // Base case 1:
8      // position ligger udenfor mazen
9      if (row < 0 || row >= ROWS ||
10         col < 0 || col >= COLS)
11      {
12          return false;
13      }
14
15      // Base case 2:
16      // positionen rammer en væg eller er allerede blevet tjekket
17      if (maze[row][col] == 'X' ||
18          maze[row][col] == 'V')
19      {
20          return false;
21      }
22
23      // Base case 3:
24      // Exit er fundet
25      if (maze[row][col] == 'E')
26      {
27          return true;
28      }
29
30      // Marker felt som tjekket
31      maze[row][col] = 'V';
32
33      // Rekursiv case:
34      // Tjek op, ned, venstre, højre
35      return findPath(maze, row - 1, col) ||
36             findPath(maze, row + 1, col) ||
37             findPath(maze, row, col - 1) ||
38             findPath(maze, row, col + 1);
39  }
```

```

40
41 int main()
42 {
43     // Valid maze
44     char maze1[ROWS][COLS] = {
45         {'X', 'X', 'X', 'X', 'X'},
46         {'X', ' ', ' ', ' ', 'X'},
47         {'X', ' ', 'X', ' ', 'X'},
48         {'X', ' ', 'X', ' ', 'X'},
49         {'X', 'E', 'X', 'X', 'X'}
50     };
51
52     // Blokeret maze
53     char maze2[ROWS][COLS] = {
54         {'X', 'X', 'X', 'X', 'X'},
55         {'X', ' ', ' ', ' ', 'X'},
56         {'X', 'X', 'X', ' ', 'X'},
57         {'X', ' ', 'X', ' ', 'X'},
58         {'X', 'E', 'X', 'X', 'X'}
59     };
60
61     std::cout << std::boolalpha;
62
63     std::cout << "Maze 1 path found: "
64                 << findPath(maze1, 1, 1)
65                 << std::endl;
66
67     std::cout << "Maze 2 path found: "
68                 << findPath(maze2, 1, 1)
69                 << std::endl;
70
71     return 0;
72 }

```

Code Block 18: Exercise 4 kildekode

Øvelse 5

(A) Udvid List ADT med recursive search

Vi har tilføjet `int search(const Object x)` til vores List ADT fra week 3. Funktionen returnerer det nulindeksede indeks for den første forekomst af `x`, eller `-1`, hvis værdien ikke findes. Vi har valgt at lave en offentlig `search()`-funktion, der bruger en privat recursive hjælpefunktion.

Step 1: Problemets størrelse

Problemet er at søge efter en værdi blandt N elementer. Ved hvert kald undersøges den aktuelle node, hvorefter søgningen eventuelt fortsætter i den resterende del af listen. Dermed bliver tidskompleksiteten $O(N)$.

Step 2: Base cases

Hvis `current == tail`, er der ingen elementer tilbage, og vi returnerer `-1`. Hvis `current->data == x`, returnerer vi den aktuelle position. Begge tilfælde løses uden recursion, dermed er de vores base cases.

Step 3: Det rekursive tilfælde

Hvis den aktuelle værdi ikke matcher, søger vi videre ved den næste node, som starter ved `current->next`, med positionen øget med 1. I tilfælde af, at der er flere matches i listen, er vores implementation lavet sådan, at `search()`-funktion finder det første match og returner positionen på blot dette.

```
1  template <typename Object>
2  int List<Object>::search(const Object x)
3  {
4      return search_recursive(head->next, x, 0); // Start ved foerste element
5  }
6
7  template <typename Object>
8  int List<Object>::search_recursive(Node *current, const Object x, int pos)
9  {
10     if (current == tail) // Listen er gennemsoegt
11     {
12         return -1;
13     }
14     else if (current->data == x) // Foerste match er fundet
15     {
16         return pos;
17     }
18     else
19     {
20         return search_recursive(current->next, x, pos + 1); // Soeg i resten af listen
21     }
22 }
```

Code Block 19: Rekursiv søgning i List ADT

Step 4: Fremskridt mod base case

Hvert rekursivt kald bevæger sig 1 node frem. Søgningen når dermed enten en matchende node eller `tail`. Derfor stopper rekursionen.

Test

Testen undersøger en tom liste og listen `[7, 2, 4, 6, 7, 67]`. Vi bruger `assert` til at kontrollere en manglende værdi, den første forekomst af en dublet og et match ved listens sidste position:

```

1  void test_search()
2  {
3      // test empty list
4      List<int> list;
5      std::cout << "Testing search on an empty list...\n";
6      assert(list.search(67) == -1);
7      std::cout << "Passed!\n";
8
9      // test value not present
10     list.push_back(7);
11     list.push_back(2);
12     list.push_back(4);
13     list.push_back(6);
14     list.push_back(7);
15     list.push_back(67);
16
17     std::cout << "Testing search when value is not present...\n";
18     assert(list.search(69) == -1);
19     std::cout << "Passed!\n";
20
21     // test duplicate value
22     std::cout << "Testing duplicate value - should return position of first match...\n";
23     assert(list.search(7) == 0);
24     std::cout << "Passed!\n";
25
26     // test normal match
27     std::cout << "Testing normal match...\n";
28     assert(list.search(67) == 5);
29     std::cout << "Passed!\n";
30 }

```

Code Block 20: Test af search()

Den fulde test findes i Exercise05/test_simple_list.cpp. Alle fire kontroller bestod ved kørsel.

Øvelse 6

(A) Implementer selection sort som en template

Sorter en vector ved gentagne gange at finde det mindste resterende element og flytte det til den næste position. Implementeringen skal understøtte forskellige elementtyper.

Ud fra kodeeksemplerne i insertion sort dannes en lignende templatefunktion, som kan modtage en vektor af en vilkårlig type, som understøtter sammenligning med `>`. Funktionen gennemløber den resterende del af vores vektor og finder den mindste værdi. Den mindste værdi laver et swap med den værdi, som står på `A[i]`, og derefter udføres samme loop igen, indtil hele den resterende del af vores vektor er blevet gennemgået. For at kunne lave et swap af elementer bruges `std::swap`, da den bruger move i stedet for at kopiere, hvilket er en fordel ved bl.a. `std::string` og `std::vector`, da hele indholdet derved ikke bliver kopieret over, men i stedet bare flyttet.

Derved kan vi danne vores *selectionSort*, som kan ses nedenunder:

```
1  template <class T>
2  void selectionSort(std::vector<T> &values)
3  {
4      for (std::size_t i = 0; i + 1 < values.size(); i++) //Koerer N-1 gange, da det sidste element automatisk
5          //staar korrekt, naar de oevrige er paa plads. i + 1 < size undgaar underflow ved en tom vektor.
6      {
7          std::size_t current_smallest_value_index{i}; //angiver det indeks, som vi er kommet til i for-loopet
8          //som det mindste.
9          for (std::size_t j = i + 1; j < values.size(); j++) //tjekker resten af vektor fra i + 1 til
10             //slutningen, da vi ikke skal lave om paa de allerede fundne mindste vaerdier, som vi allerede har swappet,
11             //mens vi heller ikke skal tjekke i = j, da det er det samme element.
12          {
13              if (values[current_smallest_value_index] > values[j]) //hvis det nye tal er mindre end vores
14              //nuvaerende mindste tal
15              {
16                  current_smallest_value_index = j; //indeks til det mindste tal opdateres
17              }
18          }
19          std::swap(values[i], values[current_smallest_value_index]); //efter det inderste for-loop, som finder
20          //det indeks, som den mindste vaerdi staar paa laves swap med det og indeks i, som vi er kommet til.
21      }
22  }
```

Code Block 21: Selection sort med `vector<T>` som parametre

Test

For at teste vores egen implementerede selection sort dannes der følgende testfunktion med både int, float og string. Der dannes også test med en tom vektor, samt en, som kun indeholder et element. Nedenfor ses eksemplet med int:

```
1 void test_std_vector_int()
2 {
3     std::vector<int> test_vector_not_sorted = {1, 2, 7, 8, 1, 3, 5, 2345, 3};
4
5     selectionSort(test_vector_not_sorted);
6
7     std::cout << "Own implemented selectionsort result: ";
8
9     for (auto i : test_vector_not_sorted)
10    {
11        std::cout << i << ", ";
12    }
13
14    std::vector<int> test_vector_not_sorted1 = {1, 2, 7, 8, 1, 3, 5, 2345, 3};
15
16    std::cout << std::endl
17        << "result using std::sort:          ";
18
19    std::sort(test_vector_not_sorted1.begin(), test_vector_not_sorted1.end());
20
21    for (auto i : test_vector_not_sorted1)
22    {
23        std::cout << i << ", ";
24    }
25 }
```

Code Block 22: testfunktion til int ved selection sort

I main-programmet testes alle fem tests:

```
1 Test of vector with type int:
2 Own implemented selectionsort result: 1, 1, 2, 3, 3, 5, 7, 8, 2345,
3 result using std::sort:          1, 1, 2, 3, 3, 5, 7, 8, 2345,
4
5 Test of vector with type float:
6 Own implemented selectionsort result: 1.12, 1.2, 2.555, 3, 3.12312, 5.23, 7.1231, 8.8, 2345.3,
7 result using std::sort:          1.12, 1.2, 2.555, 3, 3.12312, 5.23, 7.1231, 8.8, 2345.3,
8
9 Test of vector with type string:
10 Own implemented selectionsort result: a, abc, apple, b, ba, banana, ca, cat,
11 result using std::sort:          a, abc, apple, b, ba, banana, ca, cat,
12
13 Test of empty vector:
14 Own implemented selectionsort result:
15 result using std::sort:
16
17 Test of vector with one element:
18 Own implemented selectionsort result: 2,
19 result using std::sort:          2,
```

Code Block 23: OUTPUT fra testfunktion til selection sort

Vi kan dermed konstatere, at vores selection sort fungerer korrekt, da vi får samme resultat som ved brug af `std::sort`.

(B) Angiv tidskompleksiteten i best og worst case

I funktionen vil det ydre for-loop køre $N - 1$ gange hver gang. For hvert i gennemløber det indre loop de resterende $N - i - 1$ elementer for at finde det indeks, som den mindste værdi står på. Vores funktion

kan ikke vide, at det mindste element eksempelvis allerede står forrest, før alle de resterende elementer er sammenlignet, så det indre loop kører altid færdigt uanset inputtets rækkefølge. Antallet af sammenligninger bliver derfor:

$$(N - 1) + (N - 2) + \dots + 1 = \frac{N(N - 1)}{2}.$$

Dette gælder både for en allerede sorteret, en omvendt sorteret og en tilfældig vektor. Best case, worst case og average case er derfor alle $\Theta(N^2)$.

Øvelse 7

(A) Implementer counting sort

Vi har implementeret counting sort til en `vector<int>`. Vi definerer N som antallet af elementer. Opgaven angiver k elementer med værdier fra 0 til k , så den øvre værdigrænse svarer til N . I koden sættes k derfor til `values.size()`.

Vi tæller først forekomsterne af hver værdi i `cntVec`. Derefter gennemløbes tællevektoren i stigende rækkefølge, og hver værdi skrives tilbage så mange gange, som tællingen angiver.

```
1 void countingSort(std::vector<int> &values)
2 {
3     int k = values.size(); // Øvre værdigrænse er antal elementer
4     std::vector<int> cntVec(k + 1, 0); // Plads til værdier fra 0 til k
5
6     for (int i = 0; i < k; i++)
7     {
8         cntVec[values[i]] += 1; // Tæl forekomster af værdien
9     }
10
11     int pos = 0;
12
13     for (int i = 0; i <= k; i++)
14     {
15         for (int j = 0; j < cntVec[i]; j++)
16         {
17             values[pos] = i; // Skriv værdier tilbage i stigende rækkefølge
18             pos++;
19         }
20     }
21 }
```

Code Block 24: Implementering af counting sort

(B) Tidskompleksitet og space complexity

Generelt tager counting sort $O(N + K)$ tid, hvor N er antallet af elementer og K er den øvre værdigrænse. I opgaven er $K = N$, så tidskompleksiteten bliver $O(N)$.

Oprettelsen af tællevektoren tager $O(N)$ tid, og den første løkke gennemløber N elementer. Den sidste del gennemløber $N + 1$ mulige værdier og skriver samlet N elementer tilbage. Selvom løkkerne er indlejrede, kører den inderste løkke samlet kun N gange, fordi summen af tællingerne er N .

Tællevektoren har $N + 1$ pladser, så den ekstra space complexity er $O(N)$. De øvrige variabler bruger konstant plads.

Test

Vi tester en tom vector, et enkelt element og en usorteret vector med dubletter samt grænseværdierne 0 og N . Resultaterne kontrolleres med `assert`. Alle tre tests bestod.

```
1 void test_counting_sort()
2 {
3     std::vector<int> values;
4     countingSort(values);
5     assert(values.empty());
6
7     values = {1};
8     countingSort(values);
9     assert(values.size() == 1);
10    assert(values[0] == 1);
11
12    values = {6, 2, 0, 2, 6, 1};
```

```
13     countingSort(values);
14     std::vector<int> expected = {0, 1, 2, 2, 6, 6};
15     assert(values == expected);
16 }
```

Code Block 25: Test af counting sort

Den fulde kode og test findes i **Exercise07**.

Øvelse 8

(A) Kombiner QuickSort og Insertion Sort med en cutoff

Vi tager udgangspunkt i undervisningens `quick_sort.h` og gemmer vores ændringer i `introsort.h`. Nedenfor ses valget mellem QuickSort og Insertion Sort samt de tilføjede assertions, der kontrollerer funktionernes preconditions. Selve partitioneringen er udeladt, da den følger undervisningskoden. Vi bruger den originale `insertion_sort.h`; slut-iteratoren er ikke inkluderet, så kaldet bruger `right + 1`.

```
1  constexpr int useInsertion = 16; // Højest 16 elementer: brug Insertion Sort
2
3  template <typename Comparable>
4  int partition(vector<Comparable>& a, int left, int right) {
5      assert(a.size() <= INT_MAX); // Størrelsen skal passe til int-indeks
6      assert(left >= 0); // Startindeks maa ikke være negativt
7      assert(right >= left && right - left >= 2); // Mindst tre elementer
8      assert(static_cast<size_t>(right) < a.size()); // Slutindeks er i vektoren
9
10     // Undervisnings kode //
11 }
12
13 template <typename Comparable>
14 void introsort(vector<Comparable>& a, int left, int right) {
15     assert(useInsertion >= 2); // To elementer skal bruge Insertion Sort
16     assert(a.size() <= INT_MAX);
17     assert(left >= 0 && static_cast<size_t>(left) <= a.size());
18     assert(right >= -1 &&
19         (right == -1 || static_cast<size_t>(right) < a.size()));
20     assert(left <= right + 1); // Tillad ogsaa et tomt delomraade
21
22     if (left >= right) {
23         return; // Nul eller et element er allerede sorteret
24     }
25
26     int size = right - left + 1; // Begge ender er inkluderet
27
28     if (size <= useInsertion) {
29         insertionSort(a.begin() + left, a.begin() + right + 1); // Sorter det lille delomraade
30     } else {
31         int i = partition(a, left, right); // Placer pivot
32         introsort(a, left, i - 1); // Sort small elements
33         introsort(a, i + 1, right); // Sort large elements
34         // Pivot ved i er allerede paa sin endelige plads
35     }
36 }
37
38 template <typename Comparable>
39 void introsort(vector<Comparable>& a) {
40     assert(a.size() <= INT_MAX);
41
42     if (a.empty()) {
43         return; // En tom vector skal ikke sorteres
44     }
45
46     introsort(a, 0, static_cast<int>(a.size()) - 1); // Sorter hele vektoren
47 }
```

Code Block 26: Ændringer i introsort.h

(B) Test inputstørrelser og sammenlign forskellige cutoff-værdier

Vi tester 100, 1000, 10000 og 100000 elementer for at undersøge tiden på både små og store input. Vi bruger samme seed til at generere tilfældige tal og afprøver cutoff 4, 8, 16 og 32.

```

1  const int repetitions = 20;
2  double totalTime = 0;
3
4  for (int test = 0; test < repetitions; ++test) {
5      std::vector<int> values = original; // Samme input i hver gentagelse
6
7      auto start = std::chrono::high_resolution_clock::now();
8      introSort(values); // Kun sorteringen maales
9      auto stop = std::chrono::high_resolution_clock::now();
10
11     auto duration = std::chrono::duration_cast<
12         std::chrono::microseconds>(stop - start);
13     totalTime += duration.count(); // Samlet tid i mikrosekunder
14
15     // Kontroller resultatet efter maalingen
16     if (!std::is_sorted(values.begin(), values.end())) {
17         std::cout << "Fejl: vektoren er ikke sorteret!\n";
18         return 1;
19     }
20 }
21
22 std::cout << "N = " << size
23     << ": gennemsnit = " << totalTime / repetitions
24     << " mikrosekunder\n";

```

Code Block 27: Tidsmåling med 20 gentagelser

N	Cutoff 4	Cutoff 8	Cutoff 16	Cutoff 32
100	11,25	8,35	11,15	11,90
1000	190,35	99,20	126,00	137,80
10000	1800,30	1696,45	1365,85	1452,95
100000	9971,10	9608,15	9983,75	11789,90

Tabel 1: Gennemsnitlig tid i mikrosekunder ved 20 gentagelser.

Sorteringstiden vokser med antallet af elementer. Cutoff 8 var hurtigst ved 100, 1000 og 100000 elementer, mens cutoff 16 var hurtigst ved 10000 elementer. Vi vælger derfor cutoff 8 ud fra denne måleserie.

En lav cutoff giver flere rekursive kald, mens en høj cutoff giver mere arbejde til Insertion Sort. Der er ikke én grænse, som er hurtigst ved alle størrelser, og tiderne varierer mellem kørsler. Resultatet gælder derfor vores konkrete forsøg.

(C) Sammenlign køretiden med `std::sort`

Vi måler `stlsort.cpp` med samme input og 20 gentagelser som i B.

```
1 auto start = std::chrono::high_resolution_clock::now();
2 std::sort(values.begin(), values.end()); // Kun sorteringen måles
3 auto stop = std::chrono::high_resolution_clock::now();
```

Code Block 28: Tidsmåling af `std::sort`

N	IntroSort, cutoff 8	std::sort
100	8,35	1,15
1000	99,20	16,55
10000	1696,45	266,85
100000	9608,15	2315,50

Tabel 2: Gennemsnitlig tid i mikrosekunder ved 20 gentagelser.

I vores forsøg var `std::sort` hurtigere end vores IntroSort ved alle fire inputstørrelser. Ved 100000 elementer brugte `std::sort` ca. 2316 mikrosekunder, mens vores IntroSort med cutoff 8 brugte ca. 9608 mikrosekunder. Vores implementation tog dermed ca. 4,1 gange så lang tid. Tiderne varierer mellem kørsler, men `std::sort` var hurtigere ved alle fire størrelser i denne sammenligning.